

Manual Técnico: Analizador Sintáctico LL

Desarrollador: Javier Alejandro Mérida Gómez - 202230638

Repositorio: https://github.com/JavierMeridaLk/PracticaDosCompiladores1_2026

1. Descripción General

La aplicación es una plataforma web avanzada diseñada para la creación, validación y análisis de gramáticas bajo el estándar LL. Utilizando el lenguaje de definición **Wison**, el sistema automatiza procesos de la teoría de compiladores como el cálculo de conjuntos de predicción, la detección de recursividad y la generación de tablas de análisis sintáctico. La herramienta permite visualizar el proceso mediante árboles de derivación interactivos y reportes técnicos de errores.

2. Requisitos del Sistema

- **Plataforma:** Sistema operativo Windows o Linux (Ubuntu recomendado).
 - **Navegador:** Compatible con estándares modernos (Chrome, Firefox, Edge).
 - **Entornos de Desarrollo (IDE):**
 - **Visual Studio Code:** Para el desarrollo de la lógica y gramática.
 - **Lenguajes y Herramientas:**
 - **Frontend:** Vue 3 (Composition API) con Vite.
 - **Backend:** Node.js (Express) para el servidor de API.
 - **Compilación:** Jison para el procesamiento del meta-lenguaje Wison.
 - **Visualización:** D3.js para la renderización de grafos (árboles).
-

3. Arquitectura del Sistema

El sistema opera bajo una arquitectura de microservicios desacoplados. El frontend gestiona la persistencia de sesión y la edición de código, mientras que el backend actúa como un motor de cómputo puro que recibe texto plano y devuelve estructuras JSON complejas (AST, Tablas y Árboles).

4. Descripción de Clases y Métodos (Backend)

A. AnalizadorTexto.js

Es el controlador central del motor de compilación.

- **analizar(contenido):** Orquestador principal. Limpia estados previos, llama al parser de Jison y gestiona la validación semántica de la gramática Wison.

- **validarGramaticaBasica(ast)**: Escanea las producciones en busca de **Recursividad Izquierda Directa**. Si un No Terminal aparece como el primer símbolo de su propia producción, detiene el proceso para evitar bucles en el motor LL.
- **tokenizar(cadena, lexico)**: Crea un analizador léxico dinámico. Toma las definiciones de terminales del usuario y genera expresiones regulares para segmentar la cadena de prueba.
- **parsearLL(tokens, tablaM, inicio)**: Implementa el algoritmo de la **Pila Predictiva**. Utiliza la Tabla M para procesar los tokens y construye el árbol de derivación que se enviará al frontend.

B. MotorLL.js

Encapsula la lógica matemática de la gramática.

- **calcularFirst()**: Algoritmo iterativo que encuentra el conjunto de terminales iniciales para cada No Terminal.
- **calcularFollow()**: Calcula los símbolos que pueden seguir a un No Terminal, considerando producciones vacías (\$\epsilon\$).
- **construirTablaM()**: Genera la matriz de análisis. Si una combinación de [NoTerminal, Terminal] intenta asignar dos producciones distintas, reporta un **Conflicto de Ambigüedad (No es LL1)**.

C. GestorArchivos.js / fileUtils.js

- **subirArchivo**: Gestiona la carga de archivos .txt al servidor, asegurando nombres únicos y almacenamiento seguro en el directorio /uploads.

5. Procedimiento y Definición de Gramáticas (Wison)

El archivo **Jison** define la estructura del lenguaje Wison. El procedimiento para construir una gramática válida es el siguiente:

1. **Bloque Lex (Lex { : }):**
 - Se definen los tokens con la palabra clave Terminal.
 - Deben iniciar con \$ _.
 - Soporta operadores como * (Kleene), + (Cerradura) y ? (Opcional).
2. **Bloque Syntax (Syntax { { : }):**
 - Se declaran los No Terminales con % _.
 - Se define obligatoriamente el Initial_Sim.
 - Las producciones usan el operador <Code> e incluyen alternativas con |.

6. Registro y Manejo de Errores

El sistema categoriza los fallos para facilitar la depuración del usuario:

- **Errores Léxicos**: Identificados por Jison o el tokenizador dinámico cuando un carácter no encaja en las definiciones.
- **Errores Sintácticos**: Cuando el orden de los símbolos no cumple con la gramática definida.

- **Errores de Construcción (LL1):** Reportados por el MotorLL cuando la gramática posee recursividad izquierda, ambigüedad o requiere factorización. Cada error incluye **Lexema**, **Línea** y **Columna**.